Function and Parameter Reference: `model_Bacteremia.py`

1 Overview

`model_Bacteremia.py` is the base simulation model imported by every other script in this project. It defines two classes – `PharmacokineticModel` and `ImmuneResponse` – and one standalone function, `dual_reservoir_model`, which together describe drug exposure, host immune clearance, and bacterial population dynamics for a two-strain, two-compartment (blood and bone reservoir) infection. A bottom-of-file `__main__` block runs one demonstration simulation when the file is executed directly; it is not a function and is not used by any other script, which instead import the class and function definitions directly. This document lists every function, its arguments, what it does, and the parameter values used throughout the project – including, where applicable, the coefficient of variation (CV) used to sample that parameter in the project's Monte Carlo scripts. No equations are given here; see the project's Results/Methods write-up for the underlying mathematical relationships.

2 Class: `PharmacokineticModel`

Represents the two-drug dosing regimen (vancomycin, then linezolid) and converts it into a free-drug concentration profile over time.

2.1 `__init__(self)`

Arguments: none (besides `self`).

What it does: Sets fixed dosing and pharmacokinetic constants for both drugs as instance attributes (dose, dosing interval, course duration, volume of distribution, free/unbound fraction, and clearance). These values are imported from the centralized `PK_PARAMS` mapping in `parameters.py` and assigned as instance attributes (Table 1).

2.2 `concentration_function(self, drug_type, total_time_h, start_h=0)`

Arguments:

- `drug_type` (`str`) – `'vancomycin'` selects the vancomycin constants; any other value selects linezolid.
- `total_time_h` (`float`) – total length, in hours, of the time grid over which the concentration profile is built. Must cover the full simulated course.
- `start_h` (`float`, default `0`) – the simulation time, in hours, at which the first dose of this drug is given.

What it does: Builds a repeated-dose, first-order-elimination concentration-time profile for the selected drug on a dense grid (10 points per hour), by superposing one decaying exponential per dose given between `start_h` and `start_h + duration`. Concentration is forced to zero for all times before `start_h`. Returns a single-argument callable, `conc_func(t)`, that linearly interpolates this profile at any query time `t`. This returned function is what gets passed into `dual_reservoir_model` as `van_func` or `lzd_func`.

Table 1: Fixed values set by `PharmacokineticModel.__init__`.

Parameter	Vancomycin	Linezolid	Meaning
Dose	1000 mg	600 mg	amount per administration
Interval	8 h	12 h	time between doses (q8h / q12h)
Duration	96 h (4 d)	1008 h (42 d)	total length of the drug course
V_d	50 L	45 L	volume of distribution
f_u	0.50	0.70	free (unbound, active) fraction
Clearance	5.0 L/h	6.0 L/h	total body clearance

3 Class: `ImmuneResponse`

Represents the host's bacterial clearance capacity as a constant rate, independently scaled per compartment.

3.1 `__init__(self, k_immune=0.055, eff_blood=1.0, eff_res=0.5)`

Arguments:

- `k_immune` (`float`, default `0.055`) – the baseline host immune clearance rate, in h^{-1}.
- `eff_blood` (`float`, default `1.0`) – a multiplier scaling how effectively the immune response clears bacteria in the blood compartment.
- `eff_res` (`float`, default `0.5`) – the equivalent multiplier for the bone reservoir compartment. Set to one-half of `eff_blood` to represent reduced immune access to a deep-seated focus of infection.

What it does: Stores these three values as instance attributes for later use by `dual_reservoir_model`.

3.2 `compute(self, t=None)`

Arguments: `t` (optional, default `None`) – accepted but not used.

What it does: Returns `self.k_immune` unconditionally. The unused `t` argument exists purely so this method can be called with the same signature as a time-dependent function (matching how `van_func(t)`/`lzd_func(t)` are called), even though the current immune response does not actually vary with time.

4 Function: dual_reservoir_model(y, t, params, van_func, lzd_func, immune_model)

The right-hand side of the ordinary differential equation (ODE) system, written in the form consumed by the project's shared `integrate_model` solver.

Arguments:

- `y` – length-4 array/list of the current state, $[S_b,\ R_b,\ S_{res},\ R_{res}]$: sensitive and resistant *S. aureus* counts (CFU/mL) in blood (b) and the bone reservoir (res).
- `t` – the current simulation time (h). Passed through to `van_func`, `lzd_func`, and `immune_model.compute` so that time-dependent drug exposure (and, if ever implemented, a time-dependent immune response) can be evaluated at this instant.
- `params` – a dictionary of growth, drug-effect, capacity, and exchange-rate values (Table 2). Analysis scripts copy the shared `BASE_PARAMS` mapping and override only the parameter under study.
- `van_func`, `lzd_func` – callables returning free vancomycin/linezolid concentration at time `t`, normally produced by `PharmacokineticModel.concentration_function`.
- `immune_model` – an `ImmuneResponse` instance.

What it does: Computes, from the current state and the drug/immune inputs, the instantaneous rate of change of each of the four bacterial subpopulations – combining logistic growth toward a compartment-specific carrying capacity, drug-driven killing (vancomycin against sensitive strains only; linezolid against both strains, at a discounted effect in the reservoir), immune-driven killing (at a discounted rate in the reservoir), and first-order exchange of bacteria between blood and the reservoir in both directions. Non-physical negative populations are clamped to zero. Reservoir linezolid exposure is multiplied by `lzd_res_fraction`; its reservoir maximum effect is additionally scaled by $\rho_{res,S}/\rho_S$. Growth otherwise follows the ordinary logistic term for active states, allowing a detectable reservoir population to reseed blood.

The numerical integration applies a hard limit of detection (LOD) of 10 CFU/mL: a compartment that falls below the LOD is set to exactly zero. Thus, sub-LOD fractional populations cannot subsequently regrow, although a nonzero reservoir can still reseed a cleared blood compartment through the exchange term.

Returns: a length-4 list, $[\dot{S}_b,\ \dot{R}_b,\ \dot{S}_{res},\ \dot{R}_{res}]$, the derivatives the shared solver integrates forward in time.

5 Shared numerical solver

The model right-hand side does not call SciPy directly. Analysis scripts pass it to `solver.integrate_model`, which uses `scipy.integrate.solve_ivp` with LSODA. The centralized numerical configuration is relative tolerance 10^{-6}, absolute tolerance 10^{-8} CFU/mL, and maximum internal step 4.0 h. The standard output grid contains 1453 points and includes all treatment boundaries exactly.

The solver implements extinction and reactivation as hybrid events. A state crossing downward through the 10-CFU/mL LOD is set to zero and is hidden from autonomous growth and outgoing-transfer terms. Incoming transfer from an active paired compartment may accumulate until an upward crossing reaches the LOD, at which point that state is reactivated. Every returned burden

is either zero or at least 10 CFU/mL. Solver failure, warnings, non-finite output, or negative states raise an explicit integration error.

6 Script execution (`if __name__ == "__main__"`)

Not a function – this is top-level code that runs only when `model_Bacteremia.py` is executed directly (not when it is imported). It builds one `PharmacokineticModel` and one `ImmuneResponse`, defines a single baseline `params` dictionary and initial condition $Y_0 = [0, 0, 100, 100]$, integrates `dual_reservoir_model` once over a 1944-hour (81-day) course with the shared `integrate_model` routine, and prints two sanity-check values: the net pre-treatment blood growth rate (asserted positive, to catch an accidentally unrepresentative parameter set in which the untreated host would clear infection on its own) and the peak sensitive-strain blood count reached before vancomycin starts.

7 Baseline parameter values

Table 2 lists every key in the `params` dictionary consumed by `dual_reservoir_model`, with the baseline value used in this project's scripts (values are consistent across scripts except where noted).

Table 2: `params` dictionary keys used by `dual_reservoir_model`.

Key	Baseline value	Meaning
`rho_S`	1.1 h^{-1}	blood sensitive-strain growth rate
`rho_R`	0.99 h^{-1}	blood resistant-strain growth rate (10% fitness cost relative to `rho_S`)
`rho_res_S`	0.33 h^{-1}	reservoir sensitive-strain growth rate
`rho_res_R`	0.30 h^{-1}	reservoir resistant-strain growth rate (9.1% fitness cost relative to `rho_res_S`)
`Emax_v`	1.7 h^{-1}	vancomycin maximum kill rate
`EC50_V`	0.25 mg/L	vancomycin half-maximal effective concentration
`Emax_l`	1.2 h^{-1}	linezolid maximum kill rate
`EC50_L`	0.60 mg/L	linezolid half-maximal effective concentration
`B_max_blood`	6.0×10^3 CFU/mL	blood carrying capacity (shared by $S_b + R_b$)
`B_max_reservoir`	1.0×10^4 CFU/mL	reservoir carrying capacity (shared by $S_{res} + R_{res}$)
`van_res_fraction`	0.15	vancomycin reservoir tissue-penetration fraction
`lzd_res_fraction`	0.50	outcome-calibrated linezolid bone-penetration fraction within the reported 0.40–0.60 range
`f_r_b`	5.0×10^{-5} h^{-1}	reservoir → blood exchange rate
`f_b_r`	1.0×10^{-5} h^{-1}	blood → reservoir exchange rate

8 Parameters varied with a coefficient of variation

`model_Bacteremia.py` itself defines only fixed baseline values – it contains no random sampling. Three quantities are varied in the current analysis by separate Monte Carlo scripts that import `dual_reservoir_model` and call it $n = 1000$ times per script, each call using one freshly sampled value. Log-normal distributions are mean-corrected so their *mean* (not their median) equals the

baseline value shown in Table 2. The $E_{max,L}$ analysis uses the same mean-corrected log-normal parameterization, so its baseline value is likewise the distribution's arithmetic mean. Every other parameter is held fixed at baseline. Table 3 lists each sampled quantity's mean, CV, and source script.

Table 3: Monte Carlo sampling distributions used elsewhere in this project ($n = 1000$ draws each, mean-corrected).

Quantity	**Mean**	**CV**	**Source script**
$EC_{50,L}$	0.60 mg/L	0.60	`MC_ec50_lzd.py`
$E_{max,L}$	1.2 h^{-1}	0.080	`MC_emax_lzd.py`; mean-corrected log-normal
k_{immune}	0.055 h^{-1}	0.80	`MC_immune_response.py`

The pharmacodynamic and immune CVs in Table 3 are outcome-calibrated model inputs, not independently estimated measures of population variability.

The sampled means for $EC_{50,L}$, $E_{max,L}$, and k_{immune} equal their fixed baseline values; in particular, `ImmuneResponse`'s default is 0.055 h^{-1}. Initial-condition sampling is excluded from the current analysis. Within each one-factor Monte Carlo analysis, every non-target parameter remains fixed at baseline. Separate deterministic sensitivity scripts may deliberately sweep growth-independent transfer rates or another named parameter, but those sweeps do not assign those quantities a population CV.